

TAD – Tipos Abstractos de Datos

1. TAD Producto

Propósito

Representar un producto dentro del catálogo del supermercado.

Estructura lógica

Un producto está compuesto por los siguientes atributos:

- **name:** cadena de texto
- **barcode:** cadena de texto
- **category:** cadena de texto
- **expiry_date:** cadena de texto con formato **YYYY-MM-DD**
- **brand:** cadena de texto
- **price:** número real
- **stock:** número entero

Operaciones

- **crear(nombre, codigoBarra, categoria, fechaCaducidad, marca, precio, stock) → Producto**
Crea una nueva instancia de un producto con los datos proporcionados.
- **toString(producto) → cadena**
Devuelve una representación textual del producto.

Restricciones

- El **barcode** debe identificar de forma **única** a cada producto dentro del catálogo.
- El **price** debe ser mayor o igual que **0**.
- El **stock** debe ser mayor o igual que **0**.

2. TAD ListaSimple

Propósito

Almacenar productos en una **lista enlazada simple** para manejo secuencial de datos.

Estructura lógica

- Secuencia lineal de nodos enlazados.
- Cada nodo contiene un objeto de tipo **Producto**.

Operaciones

- **crear()** → **ListaSimple**
Crea una lista vacía.
- **add(lista, producto)** → **void**
Inserta un producto en la lista.
- **remove(lista, index)** → **Producto | null**
Elimina el elemento en la posición indicada.
- **getByBarcode(lista, codigoBarra)** → **Producto | null**
Busca un producto por su código de barras.
- **getSize(lista)** → **entero**
Devuelve la cantidad de elementos de la lista..

3. TAD ListaOrdenada

Propósito

Almacenar productos en una **lista enlazada ordenada**, manteniendo el orden según el código de barras.

Estructura lógica

- Secuencia lineal de nodos enlazados.
- Los nodos están **ordenados ascendentemente por código de barras**.

Operaciones

- **crear()** → **ListaOrdenada**
Crea una lista ordenada vacía.
- **add(lista, producto)** → **void**
Inserta un producto en la posición correspondiente para mantener el orden.

- **remove(lista, index) → Producto | null**
Elimina el elemento en la posición indicada.
- **getByBarcode(lista, codigoBarra) → Producto | null**
Busca un producto según su código de barras.
- **getSize(lista) → entero**
Devuelve la cantidad de elementos almacenados.

Para todo par de nodos consecutivos **a** y **b**:

a.codigoBarra ≤ b.codigoBarra

Esto garantiza que la lista permanezca ordenada.

4. TAD Árbol AVL

Propósito

Mantener productos en un **árbol binario de búsqueda auto-balanceado**, permitiendo búsquedas eficientes por nombre.

Estructura lógica

- Árbol binario de búsqueda.
- Cada nodo contiene un **Producto**.
- Cada nodo mantiene una **altura** en el conjunto

Operaciones

- **crear() → AVL**
Crea un árbol AVL vacío.
- **insert(avl, producto) → booleano**
Inserta un producto en el árbol.
- **remove(avl, producto) → booleano**
Elimina un producto del árbol.
- **searchByName(avl, nombre) → ListaSimple**
Busca productos por nombre.
- **inorder(avl) → void**
Recorre el árbol en orden.
- **generateDotFile(avl, filename) → void**
Genera un archivo DOT para visualizar el árbol.

Invariantes

- Cada nodo mantiene el **balance AVL**.

Complejidad esperada

- **insert**: $O(\log n)$.
- **remove**: $O(\log n)$.
- **searchByName**: $O(\log n + k)$ en promedio, donde **k** es el número de coincidencias encontradas.

5. TAD Árbol B

Propósito

Indexar productos según su **fecha de caducidad**, permitiendo consultas eficientes por rangos.

Estructura lógica

- Árbol B de grado **t** (en este proyecto se utiliza grado **3**).
- Cada nodo puede contener **múltiples claves ordenadas**.

Operaciones

- **crear(grado) → BTree**
Crea un árbol B con el grado indicado.
- **insert(btree, producto) → booleano**
Inserta un producto en el árbol.
- **remove(btree, producto) → booleano**
Elimina un producto del árbol.
- **search(btree, productoClave) → Producto | null**
Busca un producto específico.
- **searchByExpiryDateRange(btree, inicio, fin) → ListaSimple**
Busca productos dentro de un rango de fechas de caducidad.
- **generateDotFile(btree, filename) → void**
Genera un archivo DOT para visualizar el árbol.

Invariantes

- Las claves dentro de cada nodo están **ordenadas**.
- Los hijos de cada nodo **particionan correctamente los rangos de claves**.
- Todas las **hojas están en el mismo nivel**.

Complejidad esperada

- **insert:** $O(\log n)$.
- **remove:** $O(\log n)$.
- **search:** $O(\log n)$.
- **searchByExpiryDateRange:** $O(\log n + r)$, donde **r** es la cantidad de resultados encontrados.

6. TAD Main (composición del sistema)

Propósito

Representar el **catálogo completo de productos**, utilizando múltiples estructuras de datos para optimizar distintos tipos de consultas.

Componentes del sistema

El catálogo está compuesto por las siguientes estructuras:

- **ListaSimple**
- **ListaOrdenada**
- **Árbol AVL**
- **Árbol B**

Operaciones de alto nivel

- **agregarProducto**
- **buscarPorNombre**
- **buscarPorRangoDeCaducidad**
- **eliminarPorCodigoBarra**
- **cargarDesdeCSV**
- **visualizarArboles**

Idea clave del diseño

El sistema utiliza un enfoque de **multi-índice**, donde cada estructura de datos está optimizada para un tipo específico de operación:

- **Lista simple:** recorrido secuencial.
- **Lista ordenada:** mantenimiento del orden.
- **Árbol AVL:** búsqueda eficiente por nombre.
- **Árbol B:** consultas eficientes por rango de fechas de caducidad.